Performance Benchmarking of secp256r1 Key Generation and ECDH Shared Secret Derivation

Lucas Bixby | CS 410 | 10/16/2026

1.Introduction

The goal in this exercise was to measure how long it takes to generate secp256r1 (NIST P 256) key pairs and to derive ECDH shared secrets, and how both scale with the number of operations N = 100, 1,000, and 10,000. For my implementation I used the Python library `cryptography` , which calls OpenSSL.

2. Methodology

Environment. Benchmarks ran on a MacBook Air (Apple M2, 2022; 8 GB memory; macOS Sequoia 15.7.3) with Python 3.13.5 and the `cryptography` library 45.0.5. The machine was otherwise idle during testing, along with being plugged in.

Key generation (Task 1). For each N, the script generates N static and N ephemeral key pairs with `ec.generate_private_key(ec.SECP256R1())` . The timer covers only this loop. Average latency per key pair is total time divided by N (and per individual key, divided by 2N).

Shared secret derivation (Task 2). Alice and Bob key pairs are pre-generated before the timer starts. The timed loop calls `alice_priv.exchange(ec.ECDH(), bob_pub)` N times, so only the ECDH point multiplication is measured. Average derivation time is total time divided by N.

Timing. Elapsed time is measured with `time.perf_counter()` around each timed loop. Each batch size was run once as an unrecorded warmup, then repeated 50 times. Results are reported as the median and standard deviation (SD) over those 50 runs. The `time` command was also run once on each of the two benchmark scripts, and the resulting user + sys CPU time, which excludes I/O wait, is reported in section 3.3 within Table 3.

Scripts. `secp256r1_benchmark_task_1.py` and `secp256r1_benchmark_task_2.py` . The full source code is linked in the Appendix.

3. Results

3.1 Key generation

Table 1. Key generation time (median and SD over 50 runs).

N	Median total (s)	SD total (s)	Median per pair (ms)	SD per pair (ms)	Median per key (ms)
100	0.0039	0.00008	0.03937	0.00084	0.01969
1,000	0.0391	0.00051	0.03915	0.00051	0.01957
10,000	0.3826	0.00596	0.03826	0.00060	0.01913

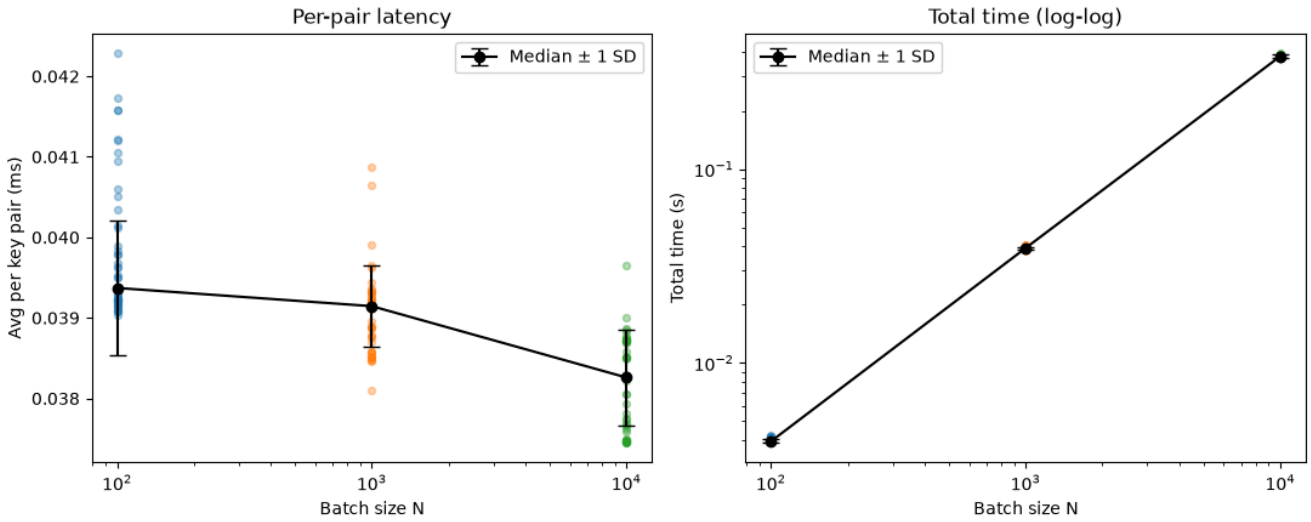


Figure 1. Key generation time versus batch size N.

Left: Average latency per key pair.

Right: Total time on log-log axes. Points are individual runs (50 per N); the black line is the median with ±1 SD error bars.

3.2 Shared secret derivation

Table 2. ECDH shared secret derivation time (median and SD over 50 runs).

N	Median total (s)	SD total (s)	Median per secret (ms)	SD per secret (ms)
100	0.0075	0.00012	0.07521	0.00124
1,000	0.0765	0.00038	0.07652	0.00038
10,000	0.7605	0.01426	0.07605	0.00143

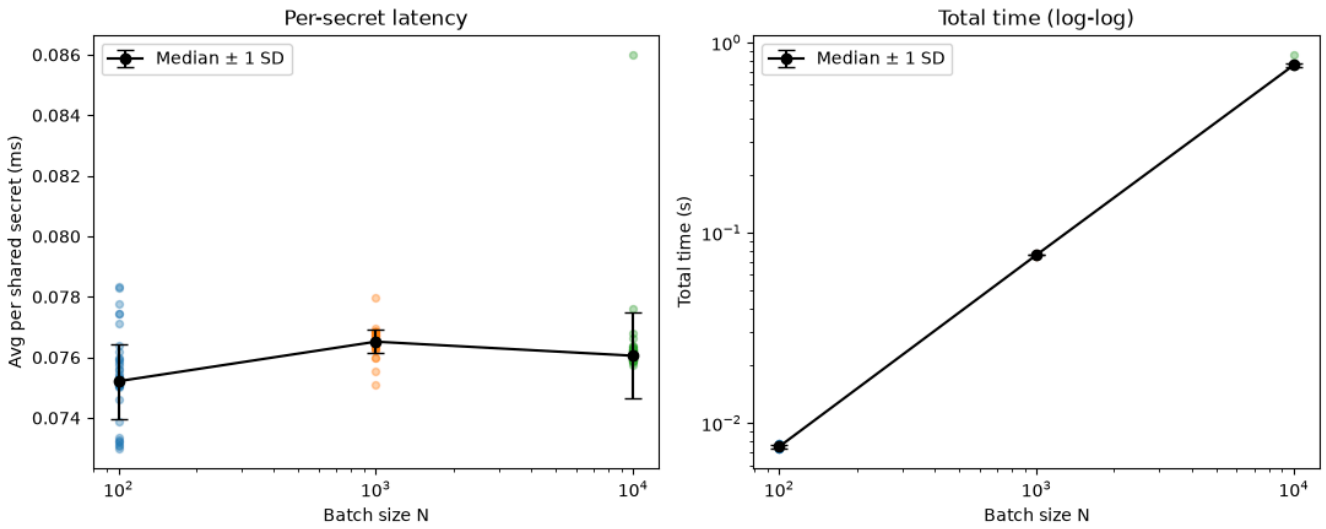


Figure 2. Shared secret derivation time versus batch size N

Left: Average latency per shared secret.

Right: Total time on log-log axes. Points are individual runs (50 per N); the black line is the median with ±1 SD error bars.

3.3 CPU time (macOS/Linux)

Table 3. User + sys CPU time from the time command. Each row covers one complete run of the script: a warmup pass, then 50 recorded runs at each of N = 100, 1,000, and 10,000, plus plotting. For derivation, this also includes the untimed pre-generation of the key pairs.

Script	User (s)	Sys (s)	User + sys (s)
Key generation	23.00	0.37	23.37
Shared secret derivation	68.70	1.54	70.24

4. Comparative Analysis

Key generation vs. derivation. Deriving one shared secret took about 3.8 to 4.0 times as long as generating one key, for example 0.0761 ms versus 0.0191 ms at N = 10,000 (medians). It also took about twice as long as generating a full static-plus-ephemeral pair (0.0383 ms). This gap is likely because key generation multiplies the fixed generator point G, which OpenSSL can speed up with precomputed tables, while ECDH multiplies an arbitrary public point.

Scaling with operation count. Total time grew in proportion to N for both operations: a 100x increase in N (100 to 10,000) raised key generation from 0.0039 s to 0.3826 s and derivation from 0.0075 s to 0.7605 s, roughly 100x each. On the log-log plots this is a straight line with slope near 1, so cost is linear, O(N). Per-operation latency stayed flat, varying by less than 3% across batch sizes.

Variability. The standard deviation over 50 runs was small relative to the median, about 0.5% to 2% at every N (for example 0.00060 ms against 0.03826 ms for key generation at N = 10,000). The spread did not shrink steadily as N grew: derivation at N = 10,000 (SD 0.01426 s, about 1.9%) was less consistent than at N = 1,000 (about 0.5%), possibly because longer runs overlap more with background activity. Running each script twice moved the medians by roughly 0.4% to 2.5%, so smaller differences should not be treated as meaningful.

5. Conclusion

Both operations scale linearly with N on this machine, at stable per-operation costs of about 0.019 ms per generated key and 0.076 ms per ECDH derivation. Derivation is roughly four times as expensive as key generation, making it the costlier step in a key exchange. At these rates a single Python thread can generate about 26,000 static-plus-ephemeral key pairs or derive about 13,000 shared secrets per second.

Appendix

Source code.
https://github.com/lucasbixby/secp256r1-ECDH-Performance-Benchmarking.git